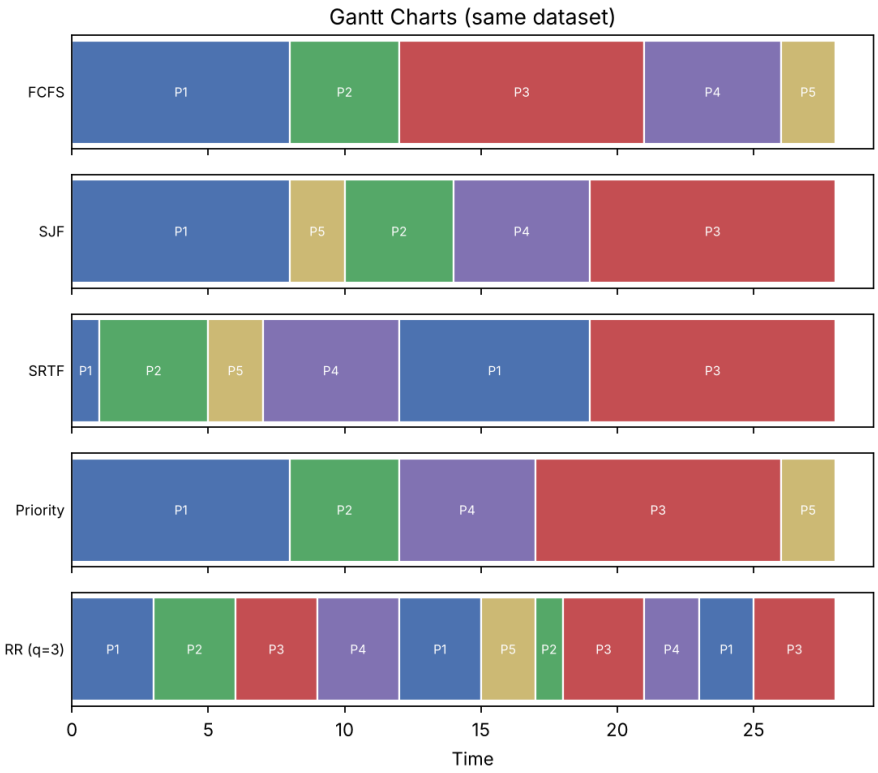
Smart CPU Scheduling Visualizer

A Python-based Operating System Resource-Management Simulator



Item	Details
Subject	Operating Systems
Technologies	Python 3, Bash, Matplotlib
Student Name	Subhrajeet Dash
Roll No	2401350018
Course	B.Tech CSE (Full Stack)
GitHub Repository	https://github.com/Subhrajeet900/OS_CAPSTONE
Submitted To	Suman Punia Ma'am
Date	13 November 2026

Contents

- Introduction and Problem Statement
- Phase 1 - System Design and Planning
- Phase 2 - Core Resource-Management Modules
- Phase 3 - Analysis and Comparison
- Phase 4 - Testing and Reliability
- Phase 5 - Documentation and Demonstration
- Conclusion and Future Scope

Introduction

An operating system has to share limited resources (CPU, memory, disk) between many programs. Choosing **how** to share them changes how fast programs finish. Textbook algorithms are hard to understand from tables alone, so this project builds a **Smart CPU Scheduling Visualizer**: a menu-driven Python program that runs OS algorithms on the same data, shows Gantt charts, and compares the results in tables and graphs.

Problem Statement

Students find it difficult to see how scheduling, page replacement, deadlock avoidance and disk scheduling actually behave. The goal is to build one simple tool that simulates these algorithms, compares them using clear metrics, handles wrong input safely, and records every action in a log file.

Objectives

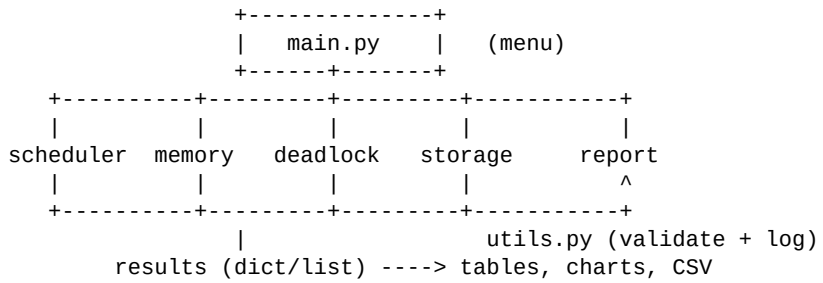
- Simulate 5 CPU scheduling algorithms and draw Gantt charts.
- Implement page replacement (FIFO, LRU), deadlock detection (Banker's), and disk scheduling (FCFS, SSTF, SCAN).
- Compare algorithms with Waiting, Turnaround and Response time and show labelled charts.
- Validate all inputs, handle errors, and keep a log file.
- Provide README, test evidence and a demo.

Phase 1 - System Design and Planning

1.1 Modules

Module	File	What it does
Process Scheduling	scheduler.py	FCFS, SJF, SRTF, Priority, Round Robin; produces Gantt data and metrics
Memory Management	memory.py	Page replacement using FIFO and LRU; counts page faults
Deadlock	deadlock.py	Banker's algorithm: checks safe state and prints safe sequence
Storage (Disk)	storage.py	Disk scheduling FCFS, SSTF, SCAN; total head movement
Reporting	report.py	Tables, matplotlib charts, saves results to CSV/PNG
Utilities	utils.py	Input validation and logging (activity.log)
Main Menu	main.py	Menu that connects all modules
Bash Script	run.sh	Checks Python, installs matplotlib, runs tests and the program

1.2 Architecture



1.3 Input / Output Flow

- **Input:** user picks a menu option, then enters data (or loads the default sample dataset).
- **Validate:** utils.py checks the numbers (positive burst time, valid priority, etc.).
- **Process:** the chosen module runs the algorithm.
- **Output:** table on screen, Gantt/bar chart saved as PNG, and a line written to activity.log.

1.4 Data Design

Data	Structure	Example
Process	Python class / tuple (pid, arrival, burst, priority)	('P1', 0, 8, 3)
Gantt chart	List of (pid, start, end)	[('P1',0,8),('P2',8,12)]
Page reference string	List of integers	[7,0,1,2,0,3]
Banker's matrices	List of lists (Allocation, Max, Available)	[[0,1,0],[2,0,0]]
Disk requests	List of cylinder numbers	[98,183,37,122]
Results	Dictionary: algorithm -> metrics	{'FCFS': (WT, TAT, RT)}

1.5 Performance Measures Used

- **Turnaround Time (TAT)** = Finish time - Arrival time
- **Waiting Time (WT)** = TAT - Burst time

- **Response Time (RT)** = First CPU start - Arrival time
- **Page faults** (memory) and **head movement** (disk)

1.6 Implementation Plan

Step	Work	Output
1	Create folder structure, utils.py (validation + logging)	Working utilities
2	Code scheduler.py with 5 algorithms	Gantt data + metrics
3	Code memory, deadlock, storage modules	3 more OS modules
4	Build menu in main.py + run.sh	Working program
5	Add report.py charts, run all datasets	Tables and graphs
6	Write tests, fix bugs	Test table
7	README, screenshots, report, demo	Final submission

Phase 2 - Core Resource-Management Modules

Five modules are implemented (the scheme requires at least three). Each algorithm is a separate reusable function, so the menu and the report module can call it with any data.

2.1 Process Scheduling (scheduler.py)

Algorithm	Idea	Type
FCFS	Run in order of arrival	Non-preemptive
SJF	Pick shortest burst among arrived	Non-preemptive
SRTF	Always run job with least remaining time	Preemptive
Priority	Smallest number = highest priority	Non-preemptive
Round Robin	Each process gets a time quantum, then goes to back of queue	Preemptive

Core code of the non-preemptive scheduler (FCFS, SJF and Priority all reuse it, only the sorting key changes):

```
def nonpre(procs, key):
    t = 0; remaining = list(procs); done = {}; gantt = []
    while remaining:
        ready = [p for p in remaining if p[1] <= t]   # arrived processes
        if not ready:                                  # CPU idle
            t = min(p[1] for p in remaining); continue
        p = min(ready, key=key)                        # pick by algorithm rule
        remaining.remove(p)
        gantt.append((p[0], t, t + p[2]))
        done[p[0]] = (p[1], p[2], t, t + p[2])         # arrival,burst,start,finish
        t += p[2]
    return done, gantt

def fcfs(procs): return nonpre(procs, lambda p: (p[1], p[0]))
def sjf(procs):  return nonpre(procs, lambda p: (p[2], p[1]))
def prio(procs): return nonpre(procs, lambda p: (p[3], p[1]))
```

2.2 Memory Management (memory.py)

Page replacement decides which page to remove from memory when all frames are full. FIFO removes the oldest page; LRU removes the page not used for the longest time.

```
def lru_page_faults(ref, frames):
    memory, faults = [], 0
    for page in ref:
        if page in memory:
            memory.remove(page); memory.append(page)   # mark as recently used
        else:
            faults += 1
            if len(memory) == frames: memory.pop(0)    # remove least recently used
            memory.append(page)
    return faults
```

2.3 Deadlock (deadlock.py)

Banker's algorithm checks if the system is in a **safe state**: is there an order in which every process can finish? If yes, it prints the safe sequence; if not, there is a deadlock risk.

```
def bankers(alloc, maximum, available):
    need = [[m - a for m, a in zip(maximum[i], alloc[i])] for i in range(len(alloc))]
    work, finished, order = available[:], [False]*len(alloc), []
    progress = True
    while progress:
        progress = False
```

```
            for i in range(len(alloc)):
                if not finished[i] and all(n <= w for n, w in zip(need[i], work)):
                    work = [w + a for w, a in zip(work, alloc[i])]   # process releases resources
                    finished[i] = True; order.append(f"P{i}"); progress = True
        return all(finished), order
```

2.4 Storage - Disk Scheduling (storage.py)

- **FCFS:** serve requests in the order they came.
- **SSTF:** always go to the nearest request.
- **SCAN (elevator):** move in one direction to the end, then reverse.

2.5 Reporting, Menu Flow and Bash Integration

report.py builds tables and saves charts. main.py shows this menu and loops until the user exits:

```
===== SMART CPU SCHEDULING VISUALIZER =====
1. CPU Scheduling (run one algorithm)
2. Compare all CPU algorithms
3. Page Replacement (FIFO / LRU)
4. Deadlock Check (Banker's)
5. Disk Scheduling
6. View activity log
0. Exit
Enter choice:
```

Bash integration: run.sh checks that Python and matplotlib exist, runs the test file, then starts the program, and 'tail activity.log' can be used to see recent actions.

```
#!/bin/bash
command -v python3 >/dev/null || { echo "Python3 not found"; exit 1; }
python3 -c "import matplotlib" 2>/dev/null || pip install matplotlib
python3 tests.py && python3 main.py
echo "Last 5 log entries:"; tail -n 5 activity.log
```

Phase 3 - Analysis and Comparison

All algorithms were run on the **same dataset** so the comparison is fair. Round Robin uses time quantum = 3.

3.1 Dataset

Process	Arrival	Burst	Priority
P1	0	8	3
P2	1	4	1
P3	2	9	4
P4	3	5	2
P5	4	2	5

Priority: smaller number means higher priority.

3.2 CPU Scheduling Results

Algorithm	Avg Waiting Time	Avg Turnaround Time	Avg Response Time
FCFS	11.4	17.0	11.4
SJF	8.2	13.8	8.2
SRTF	6.6	12.2	4.4
Priority	10.6	16.2	10.6
RR (q=3)	14.6	20.2	4.6

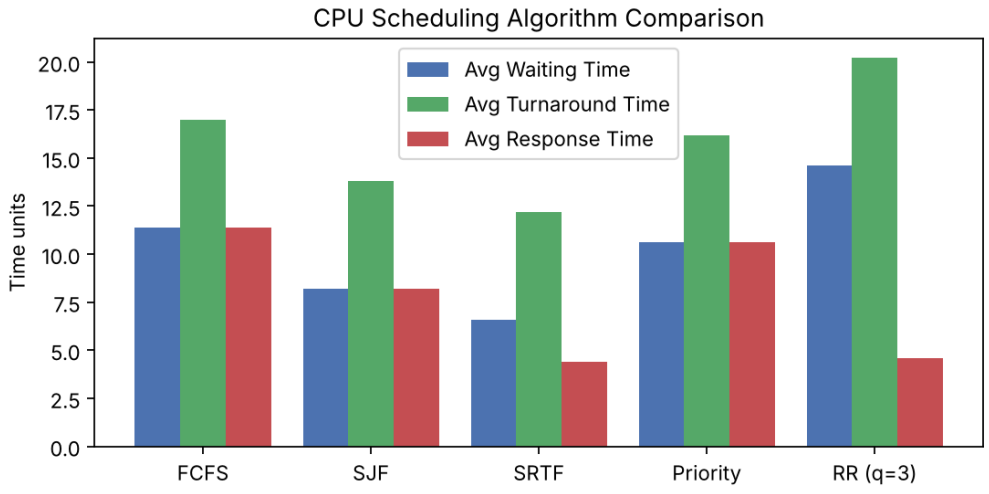


Figure 1: Average waiting, turnaround and response time

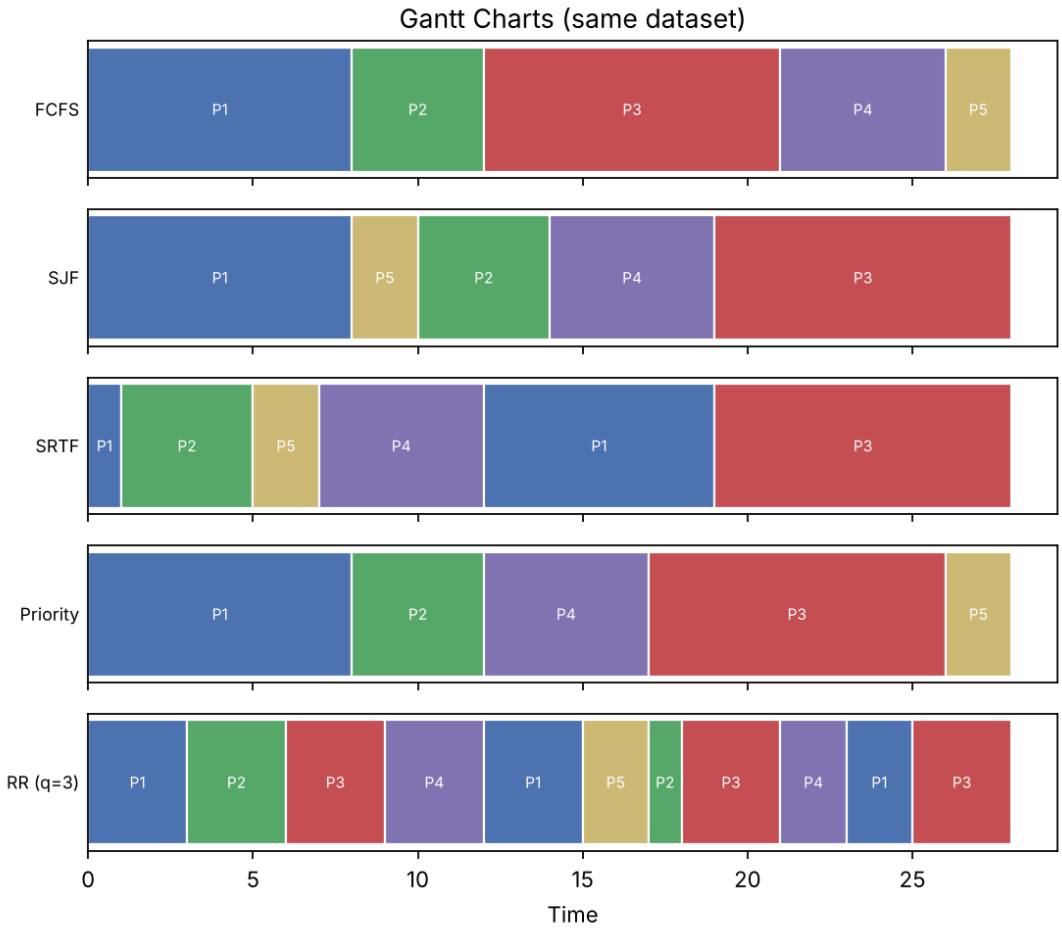


Figure 2: Gantt charts of all algorithms on the same dataset

3.3 Per-Process Detail (SJF example)

Process	Turnaround	Waiting	Response
P1	8	0	0
P2	13	9	9
P3	26	17	17
P4	16	11	11
P5	6	4	4

3.4 Memory: FIFO vs LRU (reference string of 20 pages)

Frames	2	3	4	5
FIFO faults	15	15	10	9
LRU faults	17	12	8	7

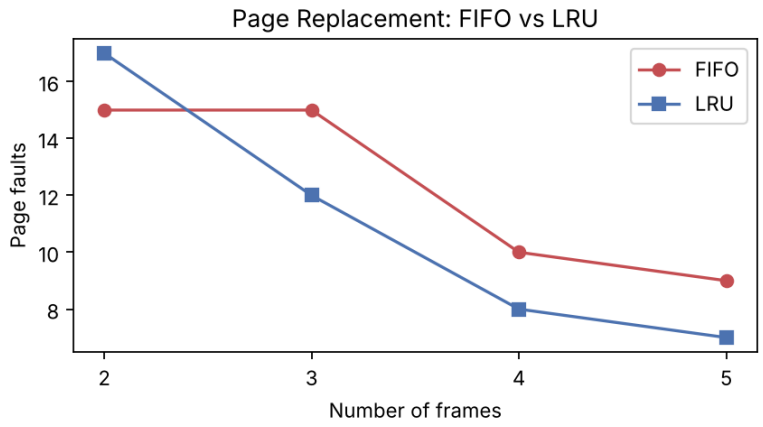


Figure 3: Page faults against number of frames

3.5 Deadlock (Banker's Algorithm)

Case	Available	Result	Safe Sequence
Standard dataset (5 processes, 3 resource types)	[3, 3, 2]	SAFE	P1 -> P3 -> P4 -> P0 -> P2
Nothing available	[0, 0, 0]	UNSAFE (deadlock risk)	none

3.6 Disk Scheduling (head at 53, requests 98,183,37,122,14,124,65,67)

Algorithm	FCFS	SSTF	SCAN
Total head movement	640	236	331

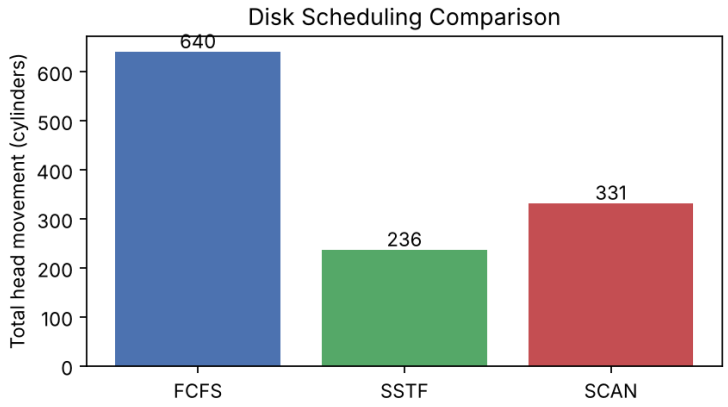


Figure 4: Total head movement

3.7 Technical Observations

- **SRTF** gave the lowest average waiting time (6.6), because it finishes short jobs first. **RR (q=3)** gave the highest (14.6).
- **FCFS** is simple but short jobs wait behind long ones (convoy effect), so its waiting time (11.4) is high.
- **Round Robin** gives a very good response time (4.6, close to the best, 4.4 by SRTF) because every process gets the CPU quickly, but it has more context switches and the largest waiting and turnaround time here.
- **SRTF** is like SJF but preemptive, so it reacts when a shorter job arrives. Its drawback is that long jobs may starve.
- **Priority** scheduling is good when some jobs matter more, but low-priority jobs can starve (solution: aging).
- **Memory:** with more frames, page faults go down for both algorithms. LRU did better than FIFO from 3 frames onwards (12 vs 15 at 3 frames, 7 vs 9 at 5), but was worse at 2 frames (17 vs 15), so neither is best in every case.
- **Disk:** SSTF (236) moves the head far less than FCFS (640), but may starve far-away requests. SCAN (331) is fairer.
- **Deadlock:** Banker's algorithm allows resource requests only when a safe sequence exists, so deadlock is avoided.

Phase 4 - Testing and Reliability

4.1 Input Validation Rules

- Number of processes must be an integer between 1 and 20.
- Arrival time must be 0 or more; burst time must be greater than 0.
- Priority must be a positive integer; time quantum must be greater than 0.
- Menu choice must be one of the listed numbers; empty input is rejected.
- Banker's matrices must have the same number of columns.

4.2 Exception Handling

```
def read_int(prompt, low, high):
    while True:
        try:
            value = int(input(prompt))
            if low <= value <= high:
                return value
            print(f"Enter a number between {low} and {high}")
        except ValueError:                # letters, empty text, decimals
            print("Invalid input. Please enter a whole number.")
            log("WARNING", "Invalid integer input")
        except (EOFError, KeyboardInterrupt):
            print("Input cancelled."); return None
```

4.3 Activity Logging

utils.py uses Python's logging module and appends every action to **activity.log** with date and time. Example:

```
2026-11-13 09:10:02 | INFO    | Program started
2026-11-13 09:10:08 | INFO    | Menu choice 2: Compare all CPU algorithms
2026-11-13 09:10:08 | INFO    | FCFS run: avg WT=11.4
2026-11-13 09:10:15 | WARNING | Invalid integer input
2026-11-13 09:10:30 | ERROR   | Burst time must be greater than 0 (got -3)
2026-11-13 09:11:00 | INFO    | Program exited normally
```

4.4 Test Cases

#	Type	Input	Expected Result	Status
1	Valid	Default dataset, FCFS	Avg WT = 11.4	Pass
2	Valid	Default dataset, SJF	Avg WT = 8.2	Pass
3	Valid	Round Robin, quantum 3	Avg RT = 4.6	Pass
4	Valid	Banker's, standard data	Safe: P1 -> P3 -> P4 -> P0 -> P2	Pass
5	Valid	Disk, head 53, SSTF	Movement = 236	Pass
6	Invalid	Burst time = -3	Error message, asks again	Pass
7	Invalid	Menu choice = 'abc'	'Invalid input', menu shown again	Pass
8	Invalid	Empty input (just Enter)	Rejected, asks again	Pass
9	Invalid	Quantum = 0	Error: quantum must be > 0	Pass
10	Boundary	Only 1 process	Runs correctly, WT = 0	Pass
11	Boundary	All arrive at time 0	Works like plain FCFS order	Pass
12	Boundary	First process arrives at time 5	CPU idle 0-5, then runs	Pass
13	Boundary	21 processes (limit 20)	Rejected with message	Pass

#	Type	Input	Expected Result	Status
14	Boundary	Frames = 1 in page replacement	Works; almost every access faults	Pass
15	Boundary	Page string with 1 page	1 fault only	Pass
16	Boundary	Banker's with 0 available	UNSAFE reported, no crash	Pass

4.5 Correctness Check

Results were verified by hand-calculating the FCFS and SJF Gantt charts and by checking that for every algorithm: (a) waiting time = turnaround - burst, (b) no process starts before it arrives, and (c) total burst time equals the total CPU busy time. An automated **tests.py** using Python's **assert** statements runs these checks every time run.sh is executed.

```
assert metrics_fcfs.avg_wt == 11.4
assert all(p.finish >= p.arrival + p.burst for p in result)
assert bankers(ALLOC, MAX, [0,0,0])[0] is False
```

Phase 5 - Documentation and Demonstration

5.1 Repository Structure

```
OS_CAPSTONE/
|-- main.py          # menu
|-- scheduler.py     # CPU scheduling
|-- memory.py        # page replacement
|-- deadlock.py      # Banker's algorithm
|-- storage.py       # disk scheduling
|-- report.py        # tables + charts
|-- utils.py         # validation + logging
|-- tests.py         # test cases
|-- run.sh           # Bash launcher
|-- sample_data/     # input datasets
|-- output/          # charts (PNG) and CSV results
|-- screenshots/     # execution evidence
|-- activity.log     # generated log
|-- README.md
```

5.2 How to Run

```
git clone https://github.com/Subhrajeet900/OS_CAPSTONE.git
cd OS_CAPSTONE
pip install matplotlib
bash run.sh          # or: python3 main.py
```

5.3 Sample Execution Output

```
Enter choice: 2

Algorithm    Avg WT   Avg TAT   Avg RT
FCFS         11.4     17.0      11.4
SJF          8.2      13.8      8.2
SRTF         6.6      12.2      4.4
Priority     10.6     16.2      10.6
RR (q=3)     14.6     20.2      4.6

Best average waiting time: SRTF
Chart saved to output/compare.png
```

5.4 Screenshots (add your own)

Take these screenshots from your own run and paste them here / in the screenshots folder as execution evidence:

- Main menu on start
- Each algorithm output with Gantt chart
- Comparison table and chart
- Invalid input being rejected
- activity.log contents
- Test run showing all tests passed

5.5 Demonstration Plan (5 minutes)

Time	What to show
0:00	Explain the problem and show the module diagram
1:00	Run option 1 (one algorithm) and show the Gantt chart
2:00	Run option 2 (compare all) and explain the chart

Time	What to show
3:00	Show page replacement, Banker's and disk scheduling quickly
4:00	Type wrong input to show error handling, then open activity.log
4:45	Show the README and conclude

5.6 Originality Statement

This project was designed and implemented by the student. Standard textbook algorithms were used as the base, but the code structure, menu, validation, logging, charts and report are the student's own work. References are listed below.

Conclusion and Future Scope

The project simulates five CPU scheduling algorithms and three other OS resource modules in one tool. The comparison shows there is no single best algorithm: SRTF gives the best waiting time, SRTF gives the best response time, and each has a cost. Input validation, exception handling and logging make the tool reliable.

Future scope: a GUI (Tkinter), more algorithms (Multilevel Queue, Optimal page replacement, C-SCAN), reading processes from CSV files, and animated Gantt charts.

References

- Silberschatz, Galvin, Gagne - *Operating System Concepts*, Wiley.
- Python documentation - logging, unittest and exceptions: docs.python.org
- Matplotlib documentation: matplotlib.org